



Progressive Education Society's
MODERN COLLEGE OF ENGINEERING, Pune -05.
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
MCA Department

PRACTICAL SUBMISSION RECORD- A.Y. 2024-25

Class: SYMCA	Div: B	Course Code: MCA10602	Batch: S1
Semester: III		Course Name: Data Science Laboratory	
Name: Tejashree Vijay Dumasia		Roll No: 52117	
CO No: CO607A.4		Assignment No: 08	

Title: Write a program for Anomaly Detection Using Autoencoder.

Code:

```
import numpy as np
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

import matplotlib.pyplot as plt

np.random.seed(42)
normal_data = np.random.normal(loc=0.0, scale=1.0, size=(1000, 20))

anomaly_data = np.random.normal(loc=4.0, scale=1.0, size=(50, 20))

X = np.vstack([normal_data, anomaly_data])
y = np.hstack([np.zeros(len(normal_data)), np.ones(len(anomaly_data))])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

input_dim = X_train.shape[1]
encoding_dim = 10

input_layer = Input(shape=(input_dim,))
encoder = Dense(encoding_dim, activation="relu")(input_layer)
decoder = Dense(input_dim, activation="linear")(encoder)
autoencoder = Model(inputs=input_layer, outputs=decoder)

autoencoder.compile(optimizer='adam', loss='mse')

autoencoder.fit(X_train[y_train == 0], X_train[y_train == 0],
                epochs=50, batch_size=32, shuffle=True, validation_split=0.1, verbose=0)

reconstructions = autoencoder.predict(X_test)
```



```
mse = np.mean(np.power(X_test - reconstructions, 2), axis=1)
```

```
threshold = np.percentile(mse, 95)
```

```
y_pred = (mse > threshold).astype(int)
```

```
print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
plt.figure(figsize=(8,4))
```

```
plt.hist(mse[y_test == 0], bins=30, alpha=0.6, label='Normal')
```

```
plt.hist(mse[y_test == 1], bins=30, alpha=0.6, label='Anomaly')
```

```
plt.axvline(threshold, color='r', linestyle='--', label='Threshold')
```

```
plt.xlabel("Reconstruction error")
```

```
plt.ylabel("Count")
```

```
plt.legend()
```

```
plt.title("Reconstruction Error Histogram")
```

```
plt.show()
```

Output:

```
7/7 ██████████ 0s 17ms/step
Confusion Matrix:
[[197  0]
 [ 2 11]]

Classification Report:
              precision    recall  f1-score   support

     0.0         0.99      1.00      0.99        197
     1.0         1.00      0.85      0.92         13

 accuracy          0.99          0.99          0.99        210
 macro avg          0.99          0.92          0.96        210
 weighted avg          0.99          0.99          0.99        210
```

